

RESUMEN COMPLETO DEL PROYECTO —

SubastUP

Documento de referencia técnica del proyecto **SubastUP**: app de subastas con backend Node/Express + Prisma + PostgreSQL, frontend Expo/React Native y un panel de administración web (WebAdmin).

Acompaña a los markdowns de detalle: `API_ENDPOINTS.md`, `CAMBIO5_f8aa32b_a_HEAD.md`, `DESPLIEGUE.md` y `RESEED_Y_DATOS.md`.

Índice

1. Visión general y arquitectura
2. Cómo se conectan las capas (flujo de una petición)
3. Árbol de archivos
4. Backend — capas y piezas principales
5. Frontend — estructura
6. WebAdmin — panel de administración
7. Base de datos (tablas)
8. Mapa de endpoints
9. Variables de entorno
10. Usuarios de prueba (seed)
11. Cómo levantar cada cosa
12. Qué cambiamos (resumen)
13. "Quiero cambiar X → tocá Y"
14. Pendientes y notas

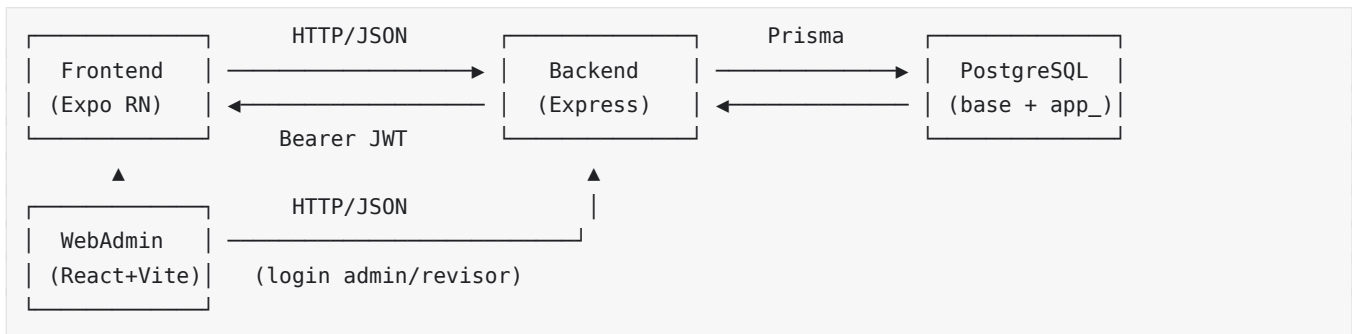
1. Visión general y arquitectura

SubastUP es una plataforma de subastas con tres componentes:

- **Backend** (`backend/`): API REST en Node + Express. Persiste en **PostgreSQL** mediante **Prisma**. Maneja autenticación JWT, el ciclo de vida de los bienes (carga → revisión → propuesta → subasta), las pujas en tiempo (timer de 60 s por ítem), pagos, notificaciones y chat.
- **Frontend** (`frontend/`): app **Expo / React Native** (corre en Android, iOS y web). Estado con **zustand**, HTTP con **axios**, navegación con **React Navigation**.

- **WebAdmin** (WebAdmin/): panel web **React + Vite** para el back-office (aprobar cuentas, verificar pagos, revisar bienes, responder mensajes). Solo entra rol `admin/revisor`.

La base sigue el modelo de la cátedra (`estructurabasica.sql`): las **16 tablas base** son inmutables; todo lo que agrega la app vive en **17 tablas con prefijo `app_`**.



2. Cómo se conectan las capas (flujo de una petición)

Ejemplo: **pujar** en una subasta.

1. El usuario toca "Pujar" en una pantalla de `frontend/src/screens/auction/`.
2. `frontend/src/services/api.js` (axios) hace `POST /api/bids` con `Authorization: Bearer <token>` y `body { auctionId, amount }`.
3. `backend/server.js` rutea a `routes/pujas.js` → `pujasController.pujar`.
4. El middleware `middleware/auth.js` valida el JWT y deja `req.user`.
5. El controlador abre una transacción con bloqueo de fila (`SELECT ... FOR UPDATE`), valida reglas de negocio (dueño no puja, categoría suficiente, pago verificado, timer vigente, importe dentro de los límites), garantiza la fila en `clientes` (vía `utils/provision.js`), crea el asistente si hace falta y registra la puja.
6. Prisma traduce a SQL contra PostgreSQL.
7. La respuesta `{ ok, importeNuevo, tiempoRestante, ... }` vuelve al frontend, que actualiza la UI.

El cierre de la subasta es **perezoso**: cuando alguien consulta `GET /api/bids/:itemId/status` y el timer expiró con pujas, el backend marca el ítem `cerrado` y determina el ganador (la puja más alta).

3. Árbol de archivos

```

TPO-DAI-SubastUP/
├── backend/                                # API REST
│   ├── server.js                          # arranque + montaje de routers
│   ├── config/
│   │   ├── prisma.js                     # cliente Prisma compartido
│   │   └── mailer.js                     # transport SMTP (Mailtrap en dev)
│   ├── middleware/auth.js                # verifica JWT → req.user
│   ├── routes/                           # 13 routers (auth, subastas, pujas, ...)
│   └── controllers/                      # lógica por dominio (10 controladores)
  
```

```

|   ├── services/                # mailService + emailTemplates
|   ├── utils/
|   |   ├── imagenes.js          # base64 ⇌ Bytes (fotos)
|   |   └── provision.js         # crea clientes/duenios (identidades de dominio)
|   └── prisma/
|       ├── schema.prisma        # modelo Prisma (base + app_*)
|       ├── seed_demo.js         # seed + reseed de datos de prueba
|       ├── seed.js, seed_fotos_demo.js # seeds legacy
|       └── *.sql                # helpers de extensiones/reconciliación
└── frontend/                    # app Expo / React Native
    └── src/
        ├── services/api.js      # cliente axios + interceptores
        ├── constants/api.js     # BASE_URL + mapa de ENDPOINTS
        ├── store/               # zustand (auth, register, settings)
        ├── navigation/          # App/Auth/Tab navigators
        ├── screens/             # auth, auction, chat, payments, profile, tabs
        ├── components/          # EstadisticasCharts, etc.
        ├── context/ThemeContext.js
        ├── hooks/useSocket.js
        └── utils/               # auctionState, images
└── WebAdmin/                    # panel admin (React + Vite)
    ├── App.jsx                 # toda la UI (4 pestañas)
    ├── main.jsx, index.html, vite.config.js
    └── package.json
└── docs/                        # esta documentación

```

4. Backend — capas y piezas principales

Arranque — `server.js`: configura CORS + JSON (límite 10mb), monta los routers bajo `/api/*`, expone `/health` y maneja 404/errores. Conecta Prisma y escucha en `process.env.PORT || 3000`.

Routers (`routes/`): cada archivo mapea rutas HTTP a funciones de un controlador y aplica `auth` donde corresponde. Los routers montados son: `auth`, `users`, `subastas` (`auctions`), `productos` (`products`), `pujas` (`bids`), `chat` (`chats`), `notificaciones`, `pagos` (`settings/payment-methods`), `settings`, `help`, `dev`. *(Existen `perfil.js` y `estadisticas.js` pero no están montados — su funcionalidad está bajo `/api/users/me*`.)*

Controladores (`controllers/`):

- `authController` — registro, login, reset de contraseña, aprobación de cuentas (con provisión de roles), asignación de categoría, listado de pendientes.
- `subastasController` — listado/calendario/detalle de subastas; deriva el estado `finalizada`.
- `productosController` — carga de bienes, "mis productos", revisión (aprobar/rechazar), propuesta al dueño.
- `pujasController` — estado de puja y registro de pujas con timer y bloqueo de fila.
- `pagosController` — métodos de pago (tarjeta/banco/cheque) y verificación.
- `notificacionesController` — notificaciones, push token, suscripción a subastas (campanita).

- `chatController` — conversaciones y mensajes.
- `perfilController / estadisticasController` — perfil y métricas de pujas.
- `devController` — reseed y visor de base (solo desarrollo).

Middleware — `auth.js`: lee `Authorization: Bearer`, verifica con `JWT_SECRET`, deja `{ registroId, personaId, email, rol, categoria }` en `req.user`.

Utils:

- `imagenes.js` — convierte `base64` ⇌ `Bytes` (las fotos se guardan como `bytea`, no como `string`).
- `provision.js` — crea de forma idempotente las identidades de dominio (`clientes + dueños`) que la base exige para pujar/cargar. **Pieza central del arreglo del flujo.**

Prisma — `config/prisma.js` exporta un único `PrismaClient`. `schema.prisma` define el modelo. `seed_demo.js` siembra los datos de prueba.

5. Frontend — estructura

- `services/api.js` — instancia `axios` con `BASE_URL`, agrega el token y centraliza llamadas.
- `constants/api.js` — `BASE_URL` (configurable por `EXPO_PUBLIC_API_URL`) y el diccionario `ENDPOINTS`.
- `store/` (zustand) — `authStore` (sesión/token), `registerStore` (alta multi-paso), `settingsStore` (preferencias/tema).
- `navigation/` — `AppNavigator` (raíz), `AuthNavigator` (login/registro/reset) y `TabNavigator` (home, calendario, etc.).
- `screens/` por dominio: `auth/`, `auction/` (lista, detalle, cargar producto, historial), `chat/`, `payments/`, `profile/`, `tabs/`.
- `utils/auctionState.js` — `normalizarEstadoSubasta(estado, cerrado)`: traduce el estado del backend a `abierta/proximamente/finalizado` para la UI (incluido el tag `"FINALIZADA"`).
- `utils/images.js` — helpers `base64` ⇌ `Image source`.

6. WebAdmin — panel de administración

App de un solo archivo (`App.jsx`) en React + Vite. `BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:3001'`. Login restringido a rol `admin/revisor`. Cuatro pestañas:

Pestaña	Qué hace	Endpoints
Usuarios	Ver pendientes, aprobar/rechazar, asignar categoría	<code>/api/auth/pendientes</code> , <code>/validate-user</code> , <code>/asignar-categoria</code>
Mensajes	Listar conversaciones y	<code>/api/chats</code> , <code>/api/chats/:id/messages</code>

	responder	
Bienes	Listar/revisar bienes, cambiar estado, aprobar propuestas o rechazar	/api/products, /api/products/:id/status, /api/products/:id/approve, /api/products/:id/reject
Métodos de pago	Ver pagos sin verificar y verificar	/api/settings/payment-methods/pending-verification, /:id/verify

7. Base de datos (tablas)

16 tablas base de la cátedra (NO se modifican): países, personas, empleados, sectores, seguros, clientes, duenios, subastadores, subastas, productos, fotos, catalogos, itemscatalogo, asistentes, pujos, registrodesubasta.

17 tablas auxiliares de la app (prefijo app_): app_registros, app_logins, app_fotosdni, app_metodospago, app_tarjetas, app_cuentasbancarias, app_cheques, app_notificaciones, app_pushtokens, app_suscripcionessubasta, app_conversaciones, app_mensajes, app_perfiles_contacto, app_productos_detalle, app_items_catalogo_detalle, app_pujos_detalle, app_devoluciones.

Relaciones clave para entender el flujo:

- Un usuario de la app es un `app_registros` (auth) ligado a una `personas`.
- Para **pujar**, la persona necesita una fila en `clientes`; para **cargar productos**, una en `duenios`. Ambas comparten PK con `personas` y referencian un `empleados` como verificador.
- Una subasta tiene `catalogos` → `itemscatalogo` (con `app_items_catalogo_detalle`: moneda, timer, cierre) → `pujos` (con `app_pujos_detalle`). El ganador es el `asistentes.cliente` de la puja más alta.

8. Mapa de endpoints

Detalle completo en `API_ENDPOINTS.md`. Resumen por área:

Área	Prefijo	Destacados
Auth	/api/auth	login, register, validate-user, pendientes, asignar-categoria
Usuarios	/api/users/me*	perfil, bids, stats, auctions
Subastas	/api/auctions	listado, calendar, today, detalle, upload-images
Productos	/api/products	mine, pending-review, listado admin, status, approve, reject, respond
Pujas	/api/bids	:itemId/status, POST / (pujar)

Chat	/api/chats	conversaciones, mensajes, create
Notificaciones	/api/notifications	listar, unread-count, subscribe (campanita)
Pagos	/api/settings/payment-methods	card/bank/check, pending-verification, verify
Dev	/api/dev	reseed, db

9. Variables de entorno

Tabla completa en `DESPLIEGUE.md`. Las principales del backend: `DATABASE_URL`, `DIRECT_URL` (Prisma), `JWT_SECRET`/`JWT_EXPIRES_IN`, `MAIL_*` (SMTP), `PORT`, `DEV_KEY`. El frontend usa `EXPO_PUBLIC_API_URL`; el WebAdmin usa `VITE_API_URL`.

10. Usuarios de prueba (seed)

Detalle en `RESEED_Y_DATOS.md`. Resumen:

Cuenta	Password	Rol/Categoría	Sirve para
admin@subastup.com	Admin1234	admin	back-office / WebAdmin
revisor@subastup.com	Revisor1234	revisor	back-office
vendedor@subastup.com	Demo1234	platino	dueño de todos los productos
demo1...demo6@subastup.com	Demo1234	comun → platino	postores (flujo completo)
sinpago@subastup.com	Demo1234	especial	pago sin verificar (no puede pujar)
pendiente / rechazado / bloqueado	Demo1234	—	casos de login bloqueado

El seed crea además **14 subastas** (10 abiertas, 2 programadas, 2 finalizadas), **3 bienes pendientes** y **2 conversaciones** de ejemplo.

11. Cómo levantar cada cosa

Guía paso a paso en `DESPLIEGUE.md`. En una línea por pieza:

- **Base local:** `docker run ... postgres:16` (puerto 5433).

- **Backend:** `cd backend && npm install && npx prisma generate && npx prisma db push && node prisma/seed_demo.js && npm start` → `http://localhost:3001`.
- **Frontend:** `cd frontend && npm install && npx expo start --clear`.
- **WebAdmin:** `cd WebAdmin && npm install && npm run dev` → `http://localhost:5173`.

Para volver a **Supabase + Railway/Render**: solo se cambian las variables de entorno (`DATABASE_URL/DIRECT_URL` al pooler de Supabase, y la `BASE_URL/VITE_API_URL` a la URL del backend en la nube). El `schema.prisma` no cambia.

12. Qué cambiamos (resumen)

Detalle en `CAMBIOS_f8aa32b_a_HEAD.md`. Lo esencial:

1. **Arreglo del flujo de subastas:** se provisionan clientes/duenos al aprobar una cuenta (`utils/provision.js` + transacción en `validateUser`), que era la causa de que la primera puja fallara.
2. **Backend en local** con PostgreSQL en Docker y puerto 3001.
3. **Pujas más robustas:** la regla de "otra subasta activa" ahora solo bloquea si el timer sigue corriendo.
4. **Tag "FINALIZADA"** derivado del cierre del ítem (backend + frontend).
5. **Tablas satélite renombradas** a `app_*`.
6. **Seed reescrito** + endpoints `/api/dev/reseed` y `/api/dev/db`.
7. **WebAdmin integrado** (de la rama `Vredondo`) y adaptado a los endpoints actuales.

13. "Quiero cambiar X → tocá Y"

Quiero...	Tocá...
Cambiar la base (local ⇌ Supabase)	<code>backend/.env</code> → <code>DATABASE_URL</code> y <code>DIRECT_URL</code>
Cambiar a dónde pega la app	<code>frontend/src/constants/api.js</code> (<code>BASE_URL</code>) o env <code>EXPO_PUBLIC_API_URL</code>
Cambiar a dónde pega el WebAdmin	<code>WebAdmin/App.jsx</code> (<code>BASE_URL</code>) o env <code>VITE_API_URL</code>
Agregar/editar datos de prueba	<code>backend/prisma/seed_demo.js</code>
Cambiar la duración del timer de puja	<code>TIMER_SEGUNDOS</code> en <code>backend/controllers/pujasController.js</code>
Reglas de quién puede pujar	<code>pujasController.pujar</code> (validaciones de dueño/categoría/pago)

Agregar un endpoint	crear handler en <code>controllers/</code> , ruta en <code>routes/</code> , montarlo en <code>server.js</code>
Cambiar el nombre físico de una tabla app	<code>@@map("app_...")</code> en <code>backend/prisma/schema.prisma</code>
Proteger/abrir las rutas de dev	<code>DEV_KEY</code> en <code>.env</code> o el mount de <code>/api/dev</code> en <code>server.js</code>

14. Pendientes y notas

- `GET /api/auctions/:id/share-link` lee `req.params.itemId` pero la ruta declara `:id` (responde 404; conviene unificar).
- Otros pendientes del análisis original están en `../TODO.md`.
- **Seguridad:** `backend/.env` no se versiona; en producción cambiá `DEV_KEY` o quitá el mount de `/api/dev` (la ruta de reseed **borra y recarga** la base).